ECSE 420 : Assignment 1 report

Julien Heng 261069152
Ethan Wu 261117309

Question 1

Part 1

To ensure the correctness of the matrix multiplication methods, we added a printMatrix method that allowed us to visualise the matrices. By being able to visualise the input matrices as well as the result matrix, we were able to use an online calculator to validate the result of our multiplication algorithms. That said, we validated the matrix multiplication algorithm on relatively small matrices. But, since matrix multiplication logic is independent of matrix size, matching results on smaller test cases provides strong confidence in the correctness of the parallel implementation.

Part 2

In matrix multiplication, each element of the result matrix at position (i, j) is computed by multiplying the elements of row i of the first matrix with the elements of column j of the second matrix and summing those products.

To parallelize this computation, we assigned each thread a subset of rows from the first matrix. The rows were divided into n approximately equal groups, where n is the number of threads used. Each thread was responsible for computing all the result values corresponding to its assigned rows. If the total number of rows was not evenly divisible by the number of threads, the final thread handled the remaining rows. Since each thread writes only to its own assigned rows in the result matrix, there is no overlap in memory access and no need for synchronization when writing results. After all threads complete execution, the result matrix contains the fully computed multiplication.

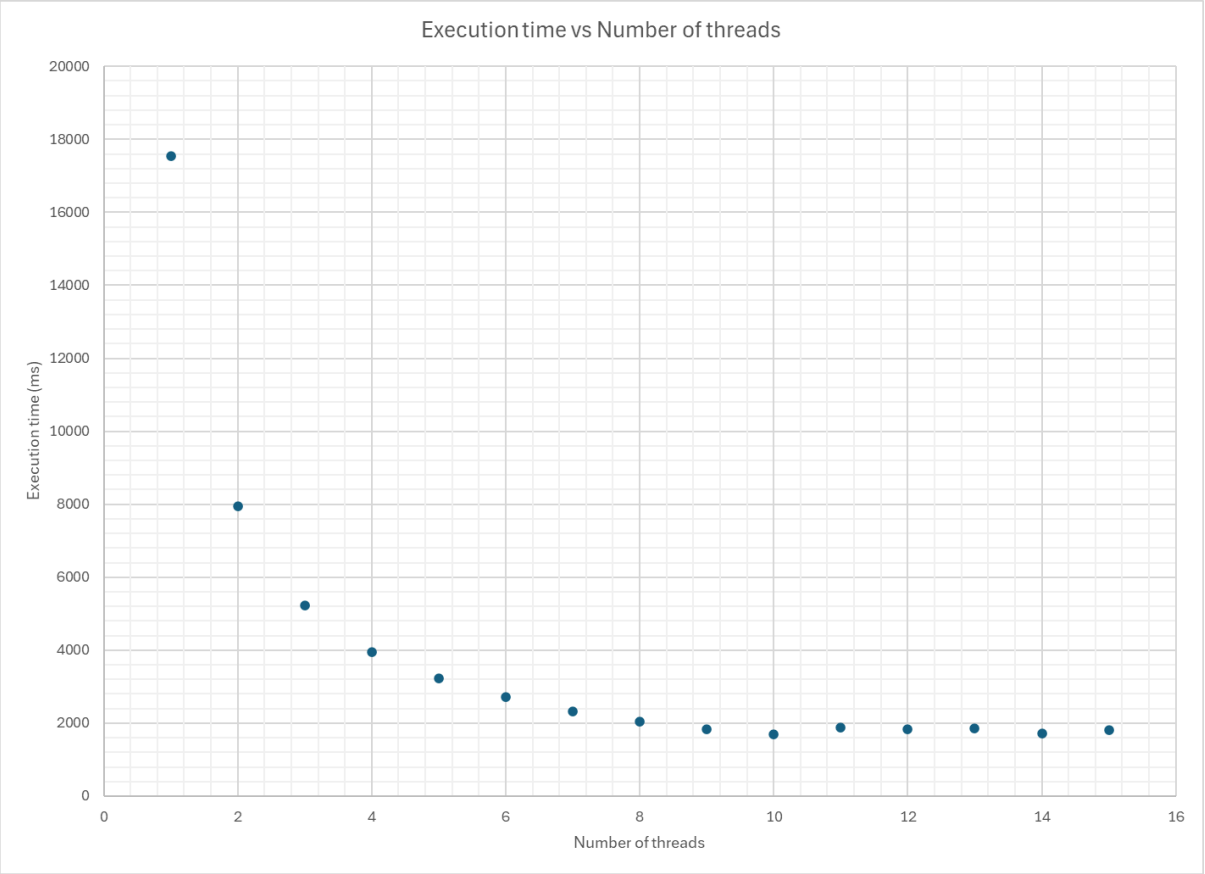
To validate the correctness of the parallelization of the multiplication, we performed both the parallel and sequential multiplication and used the printMatrix method to inspect the results and ensure the consistency of the results. Again, this was done on relatively small matrices.

Part 3

To validate that the timing results are plausible, we tested the method with progressively larger matrices and observed that the execution time increased significantly, as expected for an O(n^3) algorithm in the sequential implementation.

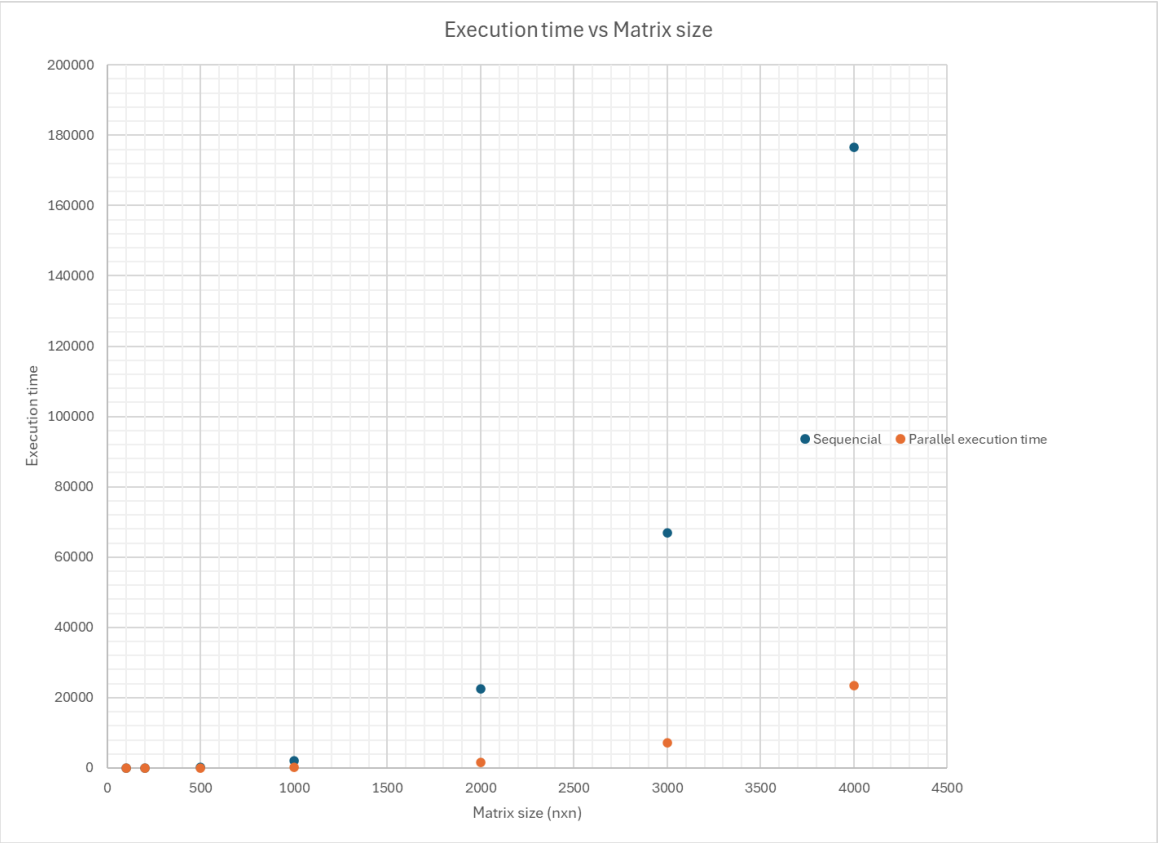
We then evaluated the parallel method using two and three threads and observed a substantial decrease in execution time, approaching approximately 2× and 3× speedup, respectively. This trend is consistent with theoretical expectations for parallel execution of independent tasks, confirming that the measured timing results are reasonable and that the parallelization behaves correctly.

Part 4



The execution time decreases approximately proportionally to the number of threads used, up to a peak speedup at 10 threads. In other words, the observed speedup is roughly equal to the number of threads for thread counts up to 10. This behavior is consistent with the hardware limitations of the system, which uses a Snapdragon X Plus processor with 10 cores (personal computer configuration). Once the number of threads exceeds the number of available cores, additional threads cannot execute truly in parallel. As a result, the execution time no longer decreases significantly and reaches a plateau.

Part 5



The sequential execution time increases much more rapidly as the matrix size grows compared to the parallel execution time. This is because matrix multiplication has time complexity O(n^3). For an n x n matrix, doubling the matrix dimension does not simply double the number of operations; instead, the number of operations increases by approximately a factor of eight. As a result, the sequential execution time grows very quickly for larger matrices.

In contrast, the parallel implementation distributes the workload across multiple threads, resulting in an approximate speedup factor of 10 on a 10-core processor. Therefore, as the matrix size increases and the sequential execution time grows substantially, the gap between the sequential and parallel execution times becomes more noticeable.

Question 2

Part 1

In general, a deadlock can occur when 2 threads working simultaneously are competing for the 2 same resources. If thread 1 has one of these resources (like a lock) and is waiting for the second resource while thread 2 has the second resource and is waiting on the first resource, it creates a deadlock and the program cannot proceed.

In our example program, the library has a book and a manga. Borrower 1 borrows the book while Borrower 2 borrows the manga. The book's lock is held by Borrower 1, and the manga's lock is held by Borrower 2, representing that these items are currently unavailable. When Borrower 1 tries to borrow the manga, it must wait for Borrower 2 to return it, while at the same time Borrower 2 tries to borrow the book. As a result, both borrowers are waiting for each other indefinitely, causing a deadlock that stops the program from progressing.

Part 2

Possible solutions to avoid deadlocks include consistent resource ordering and timeouts. With resource ordering, any thread that requires multiple resources always acquires them in the same order, preventing circular wait. In our program, this would mean that both borrowers always attempt to borrow either the book or the manga first. With timeouts, a thread stops waiting for a resource if it cannot acquire it within a certain period and releases any resources it already holds, allowing other threads to proceed. Both approaches could be applied to our program to prevent deadlock.

Question 3

Part 1

To simulate the behaviour of the Dining Philosophers problem, we model each philosopher as a thread and each chopstick as a shared object. When attempting to eat, all philosophers first pick up their left chopstick and then their right chopstick. Chopsticks are represented as objects and are used as monitors in synchronized blocks, ensuring that at most one philosopher can hold a given chopstick at any time. This enforces mutual exclusion on chopstick access. However, this setup may lead to a deadlock. If all philosophers simultaneously pick up their left chopstick before any philosopher is able to pick up their right chopstick, each philosopher will be holding one chopstick and waiting indefinitely for the other. Since no philosopher can proceed or release the chopstick they are holding, the system reaches a deadlocked state.

PS: that version of the dining philosopher code implementation is under the diningPhilosophersDeadlock.java file

Part 2

To prevent deadlocks from happening, we can change the behaviour of philosophers. Instead of having all philosophers first pick up their left chopstick and then their right chopstick, we alternate the order in which chopsticks are picked up.

For example, philosophers with even indices pick up their left chopstick first and then their right chopstick, while philosophers with odd indices pick up their right chopstick first and then their left chopstick. Therefore, there are at least two philosophers who will attempt to acquire the same chopstick first. Only one can succeed, forcing the other to wait without holding any resource. As a result, a circular chain of waiting cannot form, and deadlock is prevented.

However, this set up is not necessarily starvation-free, as one philosopher might have to wait an infinite amount of time before being able to eat. We use fair ReentrantLocks to represent chopsticks, which prioritize the philosopher that has been waiting the longest. This guarantees that no philosopher is indefinitely postponed, making the system starvation-free.

Starvation freedom can be easily tested by adding a counter for each philosopher that records the number of times the philosopher eats. If the system is starvation-free, all counters will continue to increase over time, showing that every philosopher eventually makes progress.

<u>Question 4</u>

Part 1

The sequential part of the program is 30%, therefore the parallel part of the program must be 70% ($p = 0.7$).

By Amdahl's Law, $speedup = \frac{1}{1 - p + \frac{p}{n}}$, where s is the speedup.

Then by running the program on an n-processor machine, the speedup is

$speedup = \frac{1}{1 - 0.7 + \frac{0.7}{n}} = \frac{1}{0.3 + \frac{0.7}{n}}$

If there are no bounds to the number of processors, the limit speedup is

$l = \lim_{n \to \infty} \frac{1}{0.3 + \frac{0.7}{n}}$

Since : $\lim_{n \to \infty} \frac{0.7}{n} + 0.3 = 0 + 0.3 = 0.3$

Then : $l = \lim_{n \to \infty} \frac{1}{0.3 + \frac{0.7}{n}} = \frac{1}{0.3} = 3.33$

So the limit of the speedup if there is no bound to the number of processors is 3.33.

Part 2

The sequential part of the program is 40%, therefore the parallel part is $p = 0.6$

By Amdahl's Law, we have : $S_n = \frac{1}{1 - 0.6 + \frac{0.6}{n}} = \frac{1}{0.4 + \frac{0.6}{n}}$

Similarly, for an improved version of the program where the sequential part is improved by a factor of k, the sequential part is now $\frac{0.4}{k}$ and the parallel part is $1 - \frac{0.4}{k}$

Then : $S_n' = \frac{1}{1-(1-\frac{0.4}{k})+\frac{(1-\frac{0.4}{k})}{n}} = \frac{1}{\frac{0.4}{k}+\frac{(1-\frac{0.4}{k})}{n}}$

We want to find k such that : $S_n' \geq 2S_n$

We get the following derivation :

$$\frac{1}{\frac{0.4}{k}+\frac{(1-\frac{0.4}{k})}{n}} \geq \frac{2}{0.4+\frac{0.6}{n}}$$

$$\Rightarrow 0.4 + \frac{0.6}{n} \geq 2\left(\frac{0.4}{k} + \frac{(1-\frac{0.4}{k})}{n}\right)$$

$$\Rightarrow 0.4 + \frac{0.6}{n} \geq \frac{0.8}{k} + \frac{2-\frac{0.8}{k}}{n}$$

by multiplying by n:

$$\Rightarrow 0.4n + 0.6 \geq \frac{0.8n}{k} + 2 - \frac{0.8}{k}$$

$$\Rightarrow 0.4n - 1.4 \geq \frac{0.8n-0.8}{k}$$

$$\Rightarrow k\,(0.4n - 1.4) \geq 0.8(n-1)$$

$$\Rightarrow k \geq \frac{0.8\,(n-1)}{0.4\,n-1.4}$$

$$\Rightarrow k \geq \frac{8(n-1)}{4n-14}$$

$$\Rightarrow k \geq \frac{4(n-1)}{2n-7}$$

Part 3

Suppose we have $p + s = 1$, if the sequential part can be reduced threefold we have $p' + s' = 1$ where $p' = p + \frac{2}{3}s = 1 - \frac{1}{3}s$ and $s' = \frac{1}{3}s$

And we want to find the value of s such that : $2\frac{1}{1-p+\frac{p}{n}} = \frac{1}{1-p'+\frac{p'}{n}}$

The modified program takes half the execution time of the original program on n processors. Since speedup is inversely proportional to execution time, this implies that the speedup of the modified program is twice the original speedup.

$$\Rightarrow 1 - p + \frac{p}{n} = 2\left(1 - p' + \frac{p'}{n}\right)$$

since $p' = 1 - \frac{1}{3}s$ and $p = 1 - s$

$$\Rightarrow s + \frac{1-s}{n} = 2\left(1 - \left(1 - \frac{1}{3}s\right) + \frac{1-\frac{1}{3}s}{n}\right)$$

$\Rightarrow s + \frac{1-s}{n} = \frac{2}{3}s + \frac{2-\frac{2}{3}s}{n}$

by multiplying by $3n$

$\Rightarrow 3sn + 3 - 3s = 2sn + 6 - 2s$

$\Rightarrow sn - s = 3$

$\Rightarrow s(n - 1) = 3$

$\Rightarrow s = \frac{3}{n-1}$

Appendix 1:

MatrixMultiplication

```
package ca.mcgill.ecse420.a1;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class MatrixMultiplication {

   private static final int NUMBER_THREADS = 10;
   private static final int MATRIX_SIZE = 2000;

   public static void main(String[] args) {
      // Generate two random matrices of same size
      double[][] a = generateRandomMatrix(MATRIX_SIZE, MATRIX_SIZE);
      double[][] b = generateRandomMatrix(MATRIX_SIZE, MATRIX_SIZE);

      measureTime(a, b);

   }


   /**
    * Returns the result of a sequential matrix multiplication
    * The two matrices are randomly generated
    * @param a is the first matrix
    * @param b is the second matrix
    * @return the result of the multiplication
    *
    *
    *
    * */

   public static double[][] sequentialMultiplyMatrix(double[][] a, double[][]
b) {
      double[][] result = new double[a.length][b[0].length];
```

```java
        // implement a matrix multiplication sequentially
        if (a[0].length != b.length) {
            throw new IllegalArgumentException("Matrix lengths do not match");
        }
        for (int i = 0; i < b[0].length; i++) {
            for (int j = 0; j < a.length; j++) {
                for (int k = 0; k < a[0].length; k++) {
                    result[i][j] += a[i][k] * b[k][j];
                }
            }
        }
        return result;
    }

    /**
     * Returns the result of a concurrent matrix multiplication
     * The two matrices are randomly generated
     * @param a is the first matrix
     * @param b is the second matrix
     * @return the result of the multiplication
     * */
    public static double[][] parallelMultiplyMatrix(double[][] a, double[][]
b) {
        double[][] result = new double[a.length][b[0].length];

        // each task handles a range of rows
        int rowsPerTask = a.length / NUMBER_THREADS;
        rowsPerTask = (int) Math.floor(rowsPerTask);

        List<Callable<Object>> tasks = new ArrayList<>();

        for (int i = 0; i < NUMBER_THREADS; i++) {
            int startRow = i * rowsPerTask;
            int endRow;
            // last task picks up any remaining rows
            if (i == NUMBER_THREADS - 1) {
                // take everything until the end of the array
                endRow = a.length;
            } else {
                endRow = (i + 1) * rowsPerTask;
            }
            MatrixTask aTask = new MatrixTask(a, b, result, startRow, endRow);
            startRow = endRow;

            tasks.add(Executors.callable(aTask));
        }

        ExecutorService threadPool =
Executors.newFixedThreadPool(NUMBER_THREADS);
        try {
            threadPool.invokeAll(tasks);
            threadPool.shutdown();
            threadPool.awaitTermination(1, TimeUnit.MINUTES);
```

```java
      } catch(Exception e){
         e.printStackTrace();
         System.exit(1);
      }
      return result;

   }

   /**
    * Populates a matrix of given size with randomly generated integers
between 0-10.
    * @param numRows number of rows
    * @param numCols number of cols
    * @return matrix
    */
   private static double[][] generateRandomMatrix(int numRows, int numCols) {
      double matrix[][] = new double[numRows][numCols];
      for (int row = 0; row < numRows; row++) {
         for (int col = 0; col < numCols; col++) {
            matrix[row][col] = (double) ((int) (Math.random() * 10.0));
         }
      }
      return matrix;
   }

   /**
    * Prints the given matrix to standard output.
    *
    * @param matrix the matrix to print
    */
   private static void printMatrix(double[][] matrix) {
      for (double[] row : matrix) {
         StringBuilder line = new StringBuilder();

         for (double value : row) {
            line.append(String.format("%.2f ", value));
         }

         System.out.println(line.toString().trim());
      }
   }

   /**
    * Measures and prints the execution time of both parallel and
sequential
    * matrix multiplication using the provided input matrices.
    *
    * @param a the first input matrix
    * @param b the second input matrix
    */
   private static void measureTime(double[][] a, double[][] b) {
      // measure time for parallel nultiplication
      long startTime = System.currentTimeMillis();
```

```
      double[][] result = parallelMultiplyMatrix(a, b);
      System.out.println("Parallel time with " + NUMBER_THREADS + " threads:
" + (System.currentTimeMillis() - startTime) + "ms");

      // measure time for sequential multiplication
      startTime = System.currentTimeMillis();
      result = sequentialMultiplyMatrix(a, b);
      System.out.println("sequential time: "+ (System.currentTimeMillis() -
startTime) + "ms");

   }

}
```

Appendix 2

MatrixTask

```
package ca.mcgill.ecse420.a1;


public class MatrixTask implements Runnable {

   private double[][] a;
   private double[][] b;
   private double[][] result;
   private final int startRow;
   private final int endRow;

   MatrixTask(double[][] a, double[][] b, double[][] result, int start, int
end) {

       this.a = a;
       this.b = b;
       this.result = result;

       this.startRow = start;
       this.endRow = end;
   }

   @Override
   public void run() {
       multiplyMatrix();
   }
   private void multiplyMatrix() {
       int n = a.length;
       for (int i = startRow; i < endRow; i++) { // Only loop through
assigned rows
           for (int j = 0; j < n; j++) {
               for (int k = 0; k < n; k++) {
                   result[i][j] += a[i][k] * b[k][j];
               }
           }
```

```
        }
    }
}
```

Appendix 3

Deadlock Library

```
package ca.mcgill.ecse420.a1;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.locks.*;

public class DeadlockLibrary {

   public static void main(String[] args) {
       Library lib = new Library();
       ExecutorService executor = Executors.newFixedThreadPool(2);
       executor.execute(() -> {
           Thread.currentThread().setName("Borrower1");
           lib.borrowBookandManga();
       });

       // grab in opposite order
       executor.execute(() -> {
           Thread.currentThread().setName("Borrower2");
           lib.borrowMangaandBook();
       });

       executor.shutdown();

       // Wait until all tasks are finished
       while (!executor.isTerminated()) {
       }

   }
   private static class Library {
       //fighting for these 2 ressources (locks)
       private static final Lock bookLock = new ReentrantLock();
       private static final Lock mangaLock = new ReentrantLock();


       public void borrowBookandManga() {
           bookLock.lock(); // Acquire the lock
           System.out.println(Thread.currentThread().getName() + ": Locked
Book");
           try {
               Thread.sleep(100);
           } catch (InterruptedException e) {}
           mangaLock.lock(); // try to grab mangaLock (DEADLOCK HERE)
           System.out.println(Thread.currentThread().getName() + ": Locked
Manga");

           //reading time
```

```
            mangaLock.unlock();
            bookLock.unlock(); // return the book

        }

        public void borrowMangaandBook() {
            mangaLock.lock(); // Acquire the lock
            System.out.println(Thread.currentThread().getName() + ": Locked
Manga");
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {}
            bookLock.lock(); // try to grab bookLock (DEADLOCK HERE)
            System.out.println(Thread.currentThread().getName() + ": Locked
Book");

            //reading time
            mangaLock.unlock(); //return the manga
            bookLock.unlock();
        }

    }


}
```

Appendix 4

DiningPhilosophersDeadlock

```
package ca.mcgill.ecse420.a1;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class DiningPhilosophersDeadlock {

    public static void main(String[] args) {

        int numberOfPhilosophers = 5;
        Philosopher[] philosophers = new Philosopher[numberOfPhilosophers];
        Object[] chopsticks = new Object[numberOfPhilosophers];

        for (int i = 0; i < numberOfPhilosophers; i++) {
            chopsticks[i] = new Object();
        }

        for (int i = 0; i < numberOfPhilosophers; i++) {
            philosophers[i] = new Philosopher(i, chopsticks[i], chopsticks[(i +
1) % numberOfPhilosophers]);
        }

        ExecutorService executor =
Executors.newFixedThreadPool(numberOfPhilosophers);
```

```java
        for (int i = 0; i < numberOfPhilosophers; i++) {
            executor.execute(philosophers[i]);
        }

    }

    public static class Philosopher implements Runnable {

        private int id;
        private Object leftChopstick;
        private Object rightChopstick;
        private int count;

        public Philosopher(int id, Object left, Object right) {
            this.id = id;
            this.leftChopstick = left;
            this.rightChopstick = right;
            this.count = 0;
        }

        @Override
        public void run() {
            while (true) {
                try {
                    think();
                    synchronized (leftChopstick) {
                        synchronized (rightChopstick) {
                            eat();
                        }
                    }
                } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        break;
                }
            }
        }

        private void think() throws InterruptedException {
            Thread.sleep((int) (Math.random() * 10));
        }

        private void eat() throws InterruptedException {
            count++;
            System.out.println("Philosopher " + id + " ate " + count + "
times.");
            Thread.sleep((int) (Math.random() * 10));
        }
    }

}
```

Appendix 5

DiningPhilosophers

```
package ca.mcgill.ecse420.a1;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.locks.ReentrantLock;

public class DiningPhilosophers {

   public static void main(String[] args) {

      int numberOfPhilosophers = 5;
      Philosopher[] philosophers = new Philosopher[numberOfPhilosophers];
      ReentrantLock[] chopsticks = new ReentrantLock[numberOfPhilosophers];

      for (int i = 0; i < numberOfPhilosophers; i++) {
         chopsticks[i] = new ReentrantLock(true);
      }

      for (int i = 0; i < numberOfPhilosophers; i++) {
         if (i % 2 == 0) {
            philosophers[i] = new Philosopher(i, chopsticks[i], chopsticks[(i
+ 1) % numberOfPhilosophers]);
         } else {
            philosophers[i] = new Philosopher(i, chopsticks[(i + 1) %
numberOfPhilosophers], chopsticks[i]);
         }
      }

      ExecutorService executor =
Executors.newFixedThreadPool(numberOfPhilosophers);

       for (int i = 0; i < numberOfPhilosophers; i++) {
           executor.execute(philosophers[i]);
       }

   }

   public static class Philosopher implements Runnable {

      private ReentrantLock firstChopstick;
      private ReentrantLock secondChopstick;
      private int count;
      private int id;

      public Philosopher(int id, ReentrantLock first, ReentrantLock second) {
         this.id = id;
         this.firstChopstick = first;
         this.secondChopstick = second;
         this.count = 0;
      }

      @Override
```

```
      public void run() {
         while (true) {
            try {
               think();
               firstChopstick.lock();
               try {
                  secondChopstick.lock();
                  try {
                     eat();
                  } finally {
                     secondChopstick.unlock();
                  }
               } finally {
                  firstChopstick.unlock();
               }
            } catch (InterruptedException e) {
                  Thread.currentThread().interrupt();
                  break;
            }
         }
      }

      private void think() throws InterruptedException {
           Thread.sleep((int) (Math.random() * 10));
       }

       private void eat() throws InterruptedException {
         count++;
         System.out.println("Philosopher " + id + " ate " + count + "
times.");
           Thread.sleep((int) (Math.random() * 10));
       }
   }

}
```